

HYTech LMS Developer Turnover Guide

Table of Contents

HYTech LMS Developer Turnover Guide

1. Executive summary
2. Product scope
 - 2.1 Administrator capabilities
 - 2.2 Trainer capabilities
 - 2.3 Student capabilities
 - 2.4 Important domain terminology
3. Technology stack
 - 3.1 Frontend
 - 3.2 Backend and infrastructure
 - 3.3 Testing and automation
4. Repository layout
 - 4.1 Files a new developer should read first
5. Application architecture
 - 5.1 Frontend entry and routing
 - 5.2 Authentication and authorization
 - 5.3 State and data flow
 - 5.4 Large components
6. Firebase data model
 - 6.1 Top-level collections
 - 6.2 Class subcollections
 - 6.3 Assessment authoring distinction
 - 6.4 Attempts and submissions
7. Cloud Functions
 - 7.1 Function development rules
8. Detailed setup instructions

- 8.1 Required access
- 8.2 Local prerequisites
- 8.3 Clone and enter the application
- 8.4 Install exact dependencies
- 8.5 Configure Firebase CLI aliases
- 8.6 Configure frontend environment variables
- 8.7 Configure Firebase products in a new environment
- 8.8 Create test identities and data
- 8.9 Run locally
- 8.10 Optional Firebase emulators
- 9. Testing and quality assurance
 - 9.1 Fast verification
 - 9.2 Authenticated QA configuration
 - 9.3 Required manual smoke test
 - 9.4 Rules testing
- 10. Deployment and release procedure
 - 10.1 Environment safety
 - 10.2 Staging release
 - 10.3 Production release
 - 10.4 Rollback
- 11. Security posture and known risks
 - 11.1 Protections currently present
 - 11.2 Priority unresolved risks
 - 11.3 Secrets policy
- 12. Improvement roadmap
 - Phase 0: Immediate security and release controls
 - Phase 1: Correctness and automated tests
 - Phase 2: Maintainability
 - Phase 3: Data architecture
 - Phase 4: UX, accessibility, and performance
 - Phase 5: Product and operations
- 13. Troubleshooting
 - 13.1 App says Firebase configuration is missing
 - 13.2 “Missing or insufficient permissions”
 - 13.3 Callable Function is not found
 - 13.4 Quiz submission fails
 - 13.5 Trainer cannot see or grade a manual response
 - 13.6 File upload fails
 - 13.7 Playwright refuses to run
 - 13.8 Production UI did not change

- 14. Coding and review conventions
 - 14.1 Change discipline
 - 14.2 Pull-request checklist
 - 14.3 Definition of done
- 15. Operational ownership checklist
- 16. First-week plan for a receiving developer
 - Day 1: Access and read-only orientation
 - Day 2: Reproduce the system
 - Day 3: Verify security boundaries
 - Day 4: Verify releases
 - Day 5: Start remediation
- 17. Final handover notes
- Appendix A: Command reference
- Appendix B: Environment-variable reference
- Appendix C: Useful source locations

HYTech LMS Developer Turnover Guide

System: HYTech Learning Management System

Organization: HYT Global Institute

Document purpose: Technical turnover, setup, operations, and improvement guide

Prepared: July 27, 2026

Repository application directory: [HYTech/](#)

Classification: Internal technical documentation

This guide describes the repository as it exists on the preparation date. The application, Firebase configuration, security rules, and deployment workflows should be rechecked before every production release.

1. Executive summary

HYTech LMS is a browser-based learning management system for HYT Global Institute. It supports three active application roles:

- **Administrator:** manages users, sectors, course templates, classes, system settings, logs, ID requests, and incident reports.
- **Trainer:** manages assigned classes, trainees, topics, modules, learning materials, announcements, assessments, submissions, grading, and class activity.
- **Student/Trainee:** enrolls in classes, reads learning content, submits work, takes quizzes, reviews results, receives notifications, and tracks progress.

The system is a React single-page application hosted on Firebase Hosting. It uses Firebase Authentication, Cloud Firestore, Cloud Storage, and Cloud Functions. The UI is built with React, Tailwind CSS, and Lucide icons. Vite handles local development and production builds. Playwright provides browser, role, accessibility, and responsive tests.

The backend is mostly serverless:

```
Browser
|
+--- React + React Router + Tailwind
|
+--- Firebase Authentication
+--- Cloud Firestore (application data and real-time listeners)
+--- Cloud Storage (uploaded materials, avatars, and submissions)
+--- Callable/triggered Cloud Functions (privileged and security-sensitive work)
|
Firebase Hosting serves the Vite production build
```

This repository currently distinguishes two Firebase environments:

Environment	Firebase project	Purpose
Production	hyt-global-institute-lms	Live application and real data
Staging	hytech-lms-staging	QA and safe integration testing

Never use production learner data or production credentials for local testing.

2. Product scope

2.1 Administrator capabilities

- Create, update, activate, and manage users.
- Assign administrator, trainer, and student roles.
- Manage TESDA-aligned sectors and course templates.
- Create and inspect classes.
- Preview a class from the student perspective.
- Review system activity logs.
- Process ID requests and incident reports.
- Manage global application settings and access controls.
- Read role-specific notifications.

2.2 Trainer capabilities

- View active and archived classes.
- Create classes from course templates.
- Manage class ownership and co-trainers.
- Add, approve, graduate, or remove trainees.
- Create topics, modules, materials, and announcements.
- Attach files and use announcement comments.
- Create quizzes/forms and submission-type assignments.
- Configure assessment availability, due dates, attempts, points, and passing score.
- Review quiz attempts and manually grade responses that require review.
- Grade uploaded/text/link submissions.
- View a class gradebook and activity log.
- Configure meeting links and class appearance.

2.3 Student capabilities

- Register and verify an email address.
- Sign in and access role-protected pages.
- Request or join class enrollment.
- View active and archived classes.
- Personalize class cards.
- Read announcements, topics, modules, and materials.
- Comment on announcements.
- Complete automatically graded and trainer-reviewed assessments.
- Submit text, links, and permitted file types for submission tasks.

- View task deadlines, notifications, progress, and attempt history.
- Submit an ID request or incident form.

2.4 Important domain terminology

- A **sector** groups related training programs.
- A **course template** is reusable curriculum data.
- A **class** is a live delivery instance assigned to a trainer and trainees.
- An **assessment** may be authored in either the assessment builder or the form builder.
- A **submission assignment** collects text, links, or uploaded files and is graded by a trainer.
- An **attempt** is an immutable student quiz/form response created by a Cloud Function.

3. Technology stack

3.1 Frontend

Technology	Current role
React 18	UI components and local state
React Router 7	Client-side routing and role routes
Vite 8	Development server and production bundling
Tailwind CSS 3	Responsive utility styling
Lucide React	Interface icons
Firebase Web SDK 12	Auth, Firestore, Storage, Functions, and App Check

3.2 Backend and infrastructure

Technology	Current role
Firebase Authentication	Email/password identity
Cloud Firestore	Primary application database and real-time data
Cloud Storage	User and class file storage
Cloud Functions for Firebase	Privileged writes, secure grading, triggers
Firebase Hosting	Static SPA hosting and response headers
Firebase Security Rules	Client authorization boundaries

Cloud Functions use Node.js 22 with `firebase-functions` and `firebase-admin`.

3.3 Testing and automation

Technology	Current role
Playwright	Browser and end-to-end testing
axe-core/Playwright	Accessibility checks
GitHub Actions	QA and Firebase deployment workflows

4. Repository layout

The Git repository root contains project documentation and workflow files. The deployable application lives inside `HYTech/`.

```

HYTechLMS/
|-- .github/workflows/
|   |-- qa.yml          # Build and Playwright QA
|   `-- deploy.yml      # Firebase production deployment
|-- docs/               # Architecture, QA, and roadmap documents
|-- HYTech/             # Application root; run npm/Firebase commands here
|   |-- functions/
|   |   |-- src/index.js # All Cloud Functions
|   |   `-- package.json
|   |-- public/         # Static files copied directly to dist
|   |-- scripts/        # QA provisioning, seeding, validation, cleanup
|   |-- src/
|   |   |-- components/
|   |   |   |-- admin/
|   |   |   |-- auth/
|   |   |   |-- layout/
|   |   |   |-- shared/
|   |   |   |-- student/
|   |   |   `-- trainer/
|   |   |-- context/    # Auth, toast, settings, notifications
|   |   |-- data/       # Local TESDA catalog
|   |   |-- hooks/
|   |   |-- utils/
|   |   |   `-- firestoreService.js
|   |   |-- App.jsx     # Route table
|   |   |-- firebase.js # Firebase client initialization
|   |   |-- index.css
|   |   `-- main.jsx
|   |-- tests/e2e/
|   |-- firebase.json
|   |-- firestore.rules
|   |-- firestore.indexes.json
|   |-- storage.rules
|   |-- playwright.config.js
|   |-- vite.config.js
|   `-- package.json
|-- HYTECH_LMS_DEVELOPER_TURNOVER_GUIDE.md
`-- HYTECH_LMS_DEVELOPER_TURNOVER_GUIDE.pdf

```

4.1 Files a new developer should read first

1. This turnover guide.
2. HYTech/src/App.jsx for routes and active roles.
3. HYTech/src/firebase.js for client Firebase initialization.
4. HYTech/src/utils/firestoreService.js for browser-side data operations.
5. HYTech/functions/src/index.js for trusted backend operations.
6. HYTech/firestore.rules and HYTech/storage.rules for authorization.
7. docs/FIREBASE_ENVIRONMENTS.md.
8. docs/QA_AUTOMATION.md and docs/FULL_WEBSITE_QA_PLAYBOOK.md.
9. .github/workflows/qa.yml and .github/workflows/deploy.yml.

5. Application architecture

5.1 Frontend entry and routing

`src/main.jsx` mounts the React application. `src/App.jsx` defines public and protected routes. Route protection combines Firebase Auth state, a Firestore user profile, account status, email verification, and the stored application role.

Major route families:

<code>/</code>	Public landing page
<code>/signin</code>	Sign in
<code>/signup</code>	Student registration
<code>/verify-email</code>	Email verification
<code>/admin/*</code>	Administrator application
<code>/trainer/*</code>	Trainer application
<code>/student/*</code>	Student application
<code>/class/:className</code>	Shared/legacy class entry

Unknown routes redirect to `/`.

5.2 Authentication and authorization

Authentication and authorization are deliberately separate:

- Firebase Authentication proves the identity.
- `users/{uid}` stores the application role and account status.
- Route guards control navigation and rendering.
- Firestore and Storage rules independently enforce data access.
- Cloud Functions recheck authentication, role, status, ownership, and input.

Do not rely on hidden buttons or React route guards for security. A hostile user can call Firebase APIs directly. Every sensitive operation must be protected in rules or executed through a validated Cloud Function.

New self-registrations must become students. Trainer and administrator accounts must be created or promoted through an authorized administrative workflow.

5.3 State and data flow

The system does not use Redux or another central state library. It uses:

- React component state for screen-specific state.
- `AuthContext` for authentication/profile state.

- `ToastContext` for user feedback.
- Custom hooks for settings, avatars, and notifications.
- Firestore `onSnapshot` listeners for selected real-time views.
- `firestoreService.js` as the main client data-access layer.

Typical read flow:

```
React component
  -> firestoreService helper
  -> Firestore query or subscription
  -> Security Rules authorize the request
  -> Component updates local state
```

Typical privileged write flow:

```
React component
  -> httpsCallable(...)
  -> Cloud Function validates caller and input
  -> Admin SDK transaction/batch writes Firestore
  -> Client refreshes or receives a live snapshot
```

5.4 Large components

Two components currently carry a large percentage of product behavior:

- `StudentCourse.jsx`: student class content, announcements, files, progress, quiz runner, assignment submissions, and activity.
- `ClassDetail.jsx`: trainer class management, people, content builders, responses, grading, gradebook, logs, and settings.

These components are operational but difficult to test and safely change. Their decomposition is a top maintainability priority.

6. Firebase data model

The following is a practical map, not a formal schema. Firestore documents may contain legacy fields. Always inspect production/staging data and the security rules before migrations.

6.1 Top-level collections

Collection/path	Purpose
<code>users/{uid}</code>	Public application profile, role, status, display fields

Collection/path	Purpose
users/{uid}/private/profile	Private profile fields
users/{uid}/lmsExperience/profile	LMS experience/preferences
users/{uid}/calendarEvents/{id}	Personal calendar events
users/{uid}/classPrefs/{classId}	Student class-card preferences
sectors/{sectorId}	Sector metadata
courses/{courseId}	Course-template/legacy course data
classes/{classId}	Live class metadata
classDirectory/{classId}	Sanitized class discovery record
enrollments/{id}	Student-to-class enrollment and status
notifications/{id}	Role/user notifications
activityLogs/{id}	System activity audit events
idRequests/{id}	Student ID requests
incidentForms/{id}	Student incident submissions
config/appSettings	Public branding/access configuration
students/{uid}/progress/{classId}	Per-class progress record

6.2 Class subcollections

```

classes/{classId}/
|-- members/{studentId}
|-- activity/{eventId}
|-- topics/{topicId}
|-- modules/{moduleId}
|   |-- materials/{materialId}
|-- materials/{materialId}
|-- announcements/{announcementId}
|   |-- comments/{commentId}
|-- assessments/{assessmentId}
|   |-- private/answerKey
|   |-- attempts/{attemptId}
|-- assignments/{assignmentId}
|   |-- private/answerKey
|   |-- attempts/{attemptId}
|-- submissions/{studentId}

```

6.3 Assessment authoring distinction

There are two authoring paths:

1. Assessment builder writes under

`classes/{classId}/assessments/{assessmentId}`.

2. Form/assignment builder writes under

`classes/{classId}/assignments/{assignmentId}`.

Both can produce quiz-like items. The secure submission function must resolve the item against both collections. Attempts must be read from the same parent collection that authored the item.

Correct answers belong in the private `answerKey` document, not in a trainee-readable assessment/assignment document.

6.4 Attempts and submissions

- Quiz/form attempts are written by `submitAssessmentAttempt`.
- Attempts are immutable to browser clients.
- Paragraph/manual-review attempts use `pending_review`.
- Trainers finalize reviewed attempts through `gradeAssessmentAttempt`.
- Submission assignments use a student-specific document under `submissions`.
- A resubmitted assignment must be treated as ungraded until a trainer grades the latest submission.

7. Cloud Functions

All functions are defined in `HYTech/functions/src/index.js` and deploy to the configured Firebase project. Major functions include:

Function	Type	Purpose
<code>syncClassDirectory</code>	Firestore trigger	Maintains class discovery data
<code>notifyTraineesOnAnnouncement</code>	Firestore trigger	Creates announcement notifications
<code>notifyTraineesOnAssessmentPublish</code>	Firestore trigger	Notifies on published assessments
<code>notifyTraineesOnAssignmentPublish</code>	Firestore trigger	Notifies on assignments
<code>syncClassEnrollmentCount</code>	Firestore trigger	Maintains class enrollment count
<code>migrateClassDirectory</code>	Callable	Administrative directory repair
<code>adminUpdateUserAccount</code>	Callable	Privileged Auth/profile update
<code>submitAssessmentAttempt</code>	Callable	Secure validation and grading

Function	Type	Purpose
<code>gradeAssessmentAttempt</code>	Callable	Trainer final/manual grading
<code>migrateAssessmentAnswerKeys</code>	Callable	Moves legacy answer keys
<code>promoteClassToTemplate</code>	Callable	Copies a class into a reusable template
<code>cloneTemplateToClass</code>	Callable	Copies template content into a class
<code>deleteAssessmentSecure</code>	Callable	Controlled recursive assessment deletion
<code>deleteClassSecure</code>	Callable	Controlled class deletion
<code>deleteCourseTemplateSecure</code>	Callable	Controlled template deletion
<code>deleteSectorSecure</code>	Callable	Controlled sector deletion

The file also contains enrollment/user lifecycle functions later in the module. Review the complete export list before changing deployment regions or runtime.

7.1 Function development rules

- Treat every callable input as untrusted.
- Require `context.auth` for non-public operations.
- Read the user profile and verify `status`.
- Verify role plus ownership/class trainer relationship.
- Bound strings, arrays, numeric ranges, and payload sizes.
- Use transactions for uniqueness and race-sensitive writes.
- Return safe response data only; never return private answer keys unless the assessment configuration explicitly permits it.
- Check client and server time-limit behavior together.
- Add emulator or unit tests before changing grading logic.

8. Detailed setup instructions

8.1 Required access

Before starting, request:

- Read/write access to the Git repository.
- Access to the staging Firebase project.
- Production Firebase access only if the developer is responsible for releases.

- Permission to view relevant GitHub Actions secrets/variables metadata.
- Separate staging test accounts for admin, trainer, enrolled student, and negative test cases.

Do not ask another developer to send passwords or service-account JSON through chat or commit them to Git.

8.2 Local prerequisites

Install:

1. **Git**
2. **Node.js 22 LTS**
Node 22 matches the Cloud Functions runtime.
3. **npm** bundled with Node.
4. **Firebase CLI**
5. **Playwright browsers** for end-to-end testing.

Check the tools:

```
git --version
node --version
npm --version
firebase --version
```

Recommended:

- VS Code
- Firebase/Google Cloud access with least privilege
- A password manager
- Separate browser profiles for each QA role

8.3 Clone and enter the application

```
git clone <authorized-repository-url>
Set-Location HYTechLMS
Set-Location HYTech
```

All npm and Firebase commands below assume the current directory is **HYTech/**.

8.4 Install exact dependencies

Use lockfiles for reproducible installs:

```
npm ci
npm ci --prefix functions
npx playwright install chromium
```

Install all Playwright engines only when cross-browser work is required:

```
npx playwright install
```

Do not replace `npm ci` with `npm update` during initial setup. Dependency upgrades should be isolated, reviewed, and tested.

8.5 Configure Firebase CLI aliases

The repository expects:

```
{
  "projects": {
    "default": "hyt-global-institute-lms",
    "production": "hyt-global-institute-lms",
    "staging": "hytech-lms-staging"
  }
}
```

Sign in:

```
firebase login
firebase projects:list
firebase use
```

If `.firebaserc` is unavailable, add aliases explicitly:

```
firebase use --add hytech-lms-staging
firebase use --add hyt-global-institute-lms
```

Name them `staging` and `production` when prompted.

8.6 Configure frontend environment variables

Vite exposes variables beginning with `VITE_` to browser code. Firebase web configuration values identify a Firebase project but are not authorization secrets. Security still depends on Auth, Security Rules, App Check, and backend validation.

For staging:

```
Copy-Item .env.staging.example .env.staging.local
```

Expected keys:

```
VITE_FIREBASE_API_KEY=  
VITE_FIREBASE_AUTH_DOMAIN=  
VITE_FIREBASE_PROJECT_ID=  
VITE_FIREBASE_STORAGE_BUCKET=  
VITE_FIREBASE_MESSAGING_SENDER_ID=  
VITE_FIREBASE_APP_ID=  
VITE_RECAPTCHA_SITE_KEY=
```

For the default local mode, create `.env.local` with the target project's web app configuration. Prefer staging for normal development. The repository's current default `npm run dev` reads `.env.local`; verify that file does not silently point to production.

Never commit:

- `.env.local`
- `.env.staging.local`
- `.env.e2e.local`
- service-account JSON
- Firebase refresh/CI tokens
- real user passwords

8.7 Configure Firebase products in a new environment

If reconstructing the system in a new Firebase project:

1. Create the Firebase/Google Cloud project.
2. Choose a region close to users; this project uses `asia-southeast1`.
3. Register a Firebase Web App.
4. Enable Email/Password Authentication.
5. Create the default Cloud Firestore database.
6. Create the Cloud Storage bucket.
7. Enable Firebase Hosting.
8. Enable billing before deploying Cloud Functions if required.
9. Register the production and development domains in Firebase Auth.
10. Configure reCAPTCHA v3/App Check for the web app.
11. Add environment-specific Vite values.
12. Deploy rules and indexes.
13. Deploy Functions.
14. Build and deploy Hosting.
15. Create the first administrator using a controlled bootstrap process.

Deploy infrastructure from `HYTech/`:


```
firebase deploy --only firestore:rules,firestore:indexes --project staging
firebase deploy --only storage --project staging
firebase deploy --only functions --project staging
npm run build:staging
firebase deploy --only hosting --project staging
```

Cloud Functions can require the Blaze plan. Obtain owner approval before attaching billing to a new project.

8.8 Create test identities and data

Use dedicated staging identities. Do not copy live users.

The repository contains guarded scripts:

```
npm run qa:seed:identities
npm run qa:credentials
npm run qa:seed:data
npm run qa:verify-seed
```

Read each script before running it. Confirm its project ID and dry-run behavior. Provision these minimum scenarios:

- Active administrator
- Active trainer
- Active enrolled student
- Active unenrolled student
- Pending student
- Disabled student

Each Firebase Auth identity needs a corresponding `users/{uid}` Firestore profile with the expected role and status.

8.9 Run locally

To use `.env.local`:

```
npm run dev
```

To use `.env.staging.local`:

```
npm run dev:staging
```

Vite is configured to prefer port `3000` and may select another port if it is occupied. Use the exact URL printed in the terminal.

Build and serve the production bundle locally:

```
npm run build:staging
npm run preview -- --host 127.0.0.1 --port 4173
```

8.10 Optional Firebase emulators

The current repository does not provide a complete emulator configuration or seeded emulator workflow. For backend development, add and document Auth, Firestore, Storage, and Functions emulators before relying on them.

A Functions-only command exists:

```
npm run serve -- --prefix functions
```

Functions-only emulation may still contact non-emulated services if the rest of the suite is not configured. Never assume it is isolated without verifying `FIRESTORE_EMULATOR_HOST`, Auth emulator settings, Storage emulator settings, and the active project.

9. Testing and quality assurance

9.1 Fast verification

Before committing:

```
npm run build
node --check functions/src/index.js
npm run test:e2e:public
npm run test:e2e:responsive
```

9.2 Authenticated QA configuration

Create:

```
Copy-Item .env.e2e.example .env.e2e.local
```

Populate staging-only values:

```
E2E_BASE_URL=https://hytech-lms-staging.web.app
E2E_ADMIN_EMAIL=
E2E_ADMIN_PASSWORD=
E2E_TRAINER_EMAIL=
E2E_TRAINER_PASSWORD=
E2E_STUDENT_EMAIL=
E2E_STUDENT_PASSWORD=
```

Additional negative-role variables are used by the complete QA workflow. See [.github/workflows/qa.yml](#) and the environment validator.

Run:

```
npm run test:e2e
npm run test:e2e:roles
npm run test:qa:full
```

The Playwright configuration refuses arbitrary remote targets. It allows the staging hostname, or localhost only with `E2E_ALLOW_LOCAL=true`.

9.3 Required manual smoke test

For changes affecting classes or assessments, test this complete flow:

1. Administrator creates/activates the trainer and student.
2. Trainer creates a class and enrolls/approves the student.
3. Trainer publishes an announcement and material.
4. Trainer creates an automatically graded quiz.
5. Student opens the class and submits the quiz.
6. Trainer sees the response and grade.
7. Trainer creates a paragraph/manual-review assessment.
8. Student submits it.
9. Trainer reviews and finalizes the score.
10. Trainer creates a submission assignment.
11. Student submits and resubmits.
12. Verify stale grades do not count after resubmission.
13. Verify notifications and activity events.
14. Repeat critical layouts at 320, 390, 768, 1366, and 1920 pixels.

Use the full manual playbook in [docs/FULL_WEBSITE_QA_PLAYBOOK.md](#).

9.4 Rules testing

Security Rules are production security code. At minimum, test:

- Unauthenticated denial.
- Cross-user profile denial.
- Self-registration cannot set privileged fields.
- Students cannot read private answer keys.
- Students cannot create or modify attempt grades.
- Only authorized trainers can manage a class.
- Co-trainer permissions.
- Storage MIME type, ownership, and size restrictions.
- Admin-only operations.

The project has a `scripts/test-rules.ps1` starting point, but rules testing should be expanded into deterministic emulator tests in CI.

10. Deployment and release procedure

10.1 Environment safety

Before any deploy:

```
firebase use
firebase projects:list
```

Print and verify the target project. A local build can point at a different Firebase project than the Firebase CLI deploy target; check both.

10.2 Staging release

Recommended sequence:

```
npm ci
npm ci --prefix functions
npm run build:staging
npm run test:qa:full

firebase deploy `
  --only firestore:rules,firestore:indexes,storage,functions,hosting `
  --project staging `
  --dry-run

firebase deploy `
  --only firestore:rules,firestore:indexes,storage,functions,hosting `
  --project staging
```

Run manual role smoke tests against the staging hosting URL.

10.3 Production release

Production deployment is high impact. Use a reviewed pull request, a passing QA workflow, owner approval, and a rollback plan.

Recommended sequence:

1. Confirm the change is merged from an approved branch.
2. Confirm QA passed on the exact commit.
3. Confirm database/rules migrations are backward compatible.
4. Record the current Hosting release and deployed Functions.
5. Build with production variables.
6. Run Firebase dry-run.
7. Deploy rules/indexes/functions before Hosting when the frontend depends on new backend behavior.
8. Run post-deployment smoke tests using non-destructive test accounts.
9. Monitor Functions logs, Auth errors, Firestore permission errors, and App Check metrics.
10. Roll back Hosting or redeploy the previous known-good commit if required.

Commands:

```
npm run build

firebase deploy `
  --project production `
  --dry-run `
  --non-interactive

firebase deploy `
  --project production `
  --non-interactive
```

The GitHub workflow currently deploys all configured Firebase resources on a push to `main`. See the improvement roadmap: QA must become a hard prerequisite for production deployment.

10.4 Rollback

- **Hosting:** use Firebase Hosting release history or redeploy a known-good commit.
- **Functions:** redeploy the previous known-good function code.
- **Rules:** redeploy the previous reviewed rules immediately if access breaks.
- **Firestore data:** there is no automatic rollback for destructive data mutations. Use backups/export and tested migrations.

Never use `git reset --hard`, bulk document deletion, or production migration scripts as an improvised rollback strategy.

11. Security posture and known risks

11.1 Protections currently present

- Role- and status-aware route guards.
- Firestore and Storage security rules.
- Secure callable assessment submission.
- Private assessment answer-key documents.
- Server-authorized manual grading.
- Restricted raster avatar MIME types.
- HSTS, clickjacking protection, referrer policy, permissions policy, COOP, MIME-sniffing protection, and a report-only CSP.
- Immutable caching for fingerprinted assets and no-store for `index.html`.
- Staging-host restriction in Playwright.

11.2 Priority unresolved risks

P0: Confidential security material in repository history

The repository contains an internal audit in Markdown, HTML, and PDF form. If the repository is public or broadly accessible, removing only the files in the latest commit is insufficient; earlier commits remain retrievable.

Required response:

1. Confirm repository visibility and exposure.
2. Treat exposed operational details as compromised.
3. Move internal reports to a restricted document system.
4. Use `git filter-repo` or an approved history-rewrite process.
5. Coordinate force-push and fresh clones with all contributors.
6. Rotate any credentials or sensitive values ever present.

P0/P1: Student-writable progress

Students can write progress data under enrollment and student progress paths, while trainers consume those values. Move authoritative progress calculation to trusted backend logic derived from completed attempts and submissions.

P1: Unsafe external URL protocols

Meeting links are opened from Firestore using `window.open`. Submission URLs are parsed with `new URL()`, but parsing alone accepts unsafe schemes such as `javascript:`.

Required fix:

- Permit only `https:` and, if explicitly required, `http:` in development.
- Normalize and validate on both client and trusted backend.
- Open with `noopener,noreferrer`.
- Store a validated canonical URL rather than arbitrary text.

P1: App Check is not enforced

The client initializes App Check only if `VITE_RECAPTCHA_SITE_KEY` exists. Callable Functions currently do not consistently enforce App Check, and Firebase product enforcement must also be enabled through Firebase configuration.

Roll out in stages:

1. Register all legitimate domains.
2. Configure keys per environment.
3. Verify App Check metrics and debug-token handling in staging.
4. Enforce on selected callable Functions/resources.
5. Monitor rejections before broad enforcement.

P1: QA does not block production deployment

QA and deployment workflows independently react to pushes. A failing QA run can coexist with a successful production deploy.

Required fix:

- Make deployment depend on a completed passing QA workflow or consolidate both into one workflow with `needs`.
- Use protected environments and required reviewers.
- Protect `main` and require status checks.

P1: Long-lived Firebase CI token

The deploy workflow uses `FIREBASE_TOKEN`, a long-lived credential.

Required fix:

- Prefer GitHub OpenID Connect with Google Workload Identity Federation.
- Otherwise use a narrowly scoped service account as an interim measure.
- Rotate/revoke the old token after migration.
- Use separate identities for staging and production.

P2: Answer farming

If unlimited attempts are enabled while correct answers are shown after each attempt, a student can learn the answer key and resubmit.

Possible controls:

- Show correct answers only after the due date or attempts close.
- Add a server-enforced maximum attempt count.
- Delay review.
- Draw randomized question pools.
- Record attempt policy in the assessment document and enforce it in the submission Function.

P2: CSP is report-only

Report-only mode does not block violations. Collect and review violations, remove required unsafe behavior, then promote a tested policy to **Content-Security-Policy**.

P2: Dependency and supply-chain hygiene

Review **npm audit** output carefully. Do not apply destructive forced downgrades without confirming runtime exposure. Keep frontend and Functions dependency trees separate, use lockfiles, and automate reviewed updates.

11.3 Secrets policy

Never store the following in Git, documents, screenshots, issues, or chat:

- Passwords
- Service-account JSON/private keys
- Firebase CLI refresh tokens
- App Check debug tokens
- Production exports containing personal data
- Learner records

Firebase Web API keys are embedded in browser bundles by design, but they still must be restricted appropriately in Google Cloud and never treated as the security boundary.

12. Improvement roadmap

Phase 0: Immediate security and release controls

1. Remove the confidential audit from public Git history.
2. Validate URL protocols for meetings, submissions, and external resources.
3. Replace student-writable progress with backend-derived progress.
4. Gate production deployment on passing QA and approval.
5. Replace `FIREBASE_TOKEN` with Workload Identity Federation.
6. Complete and enforce App Check rollout.
7. Add rate limiting/attempt limits to assessment submission.
8. Confirm production backups and recovery procedures.

Phase 1: Correctness and automated tests

1. Add unit tests for every assessment question type.
2. Add Functions emulator tests for:
 - one-response enforcement;
 - timed-out incomplete submissions;
 - manual grading authorization;
 - assessment-versus-assignment lookup;
 - private answer-key protection;
 - attempt limits and due dates.
3. Add Firestore and Storage Rules emulator tests.
4. Add authenticated browser tests for complete admin/trainer/student journeys.
5. Make stale-grade resubmission behavior explicit in the data model.
6. Add idempotency keys or deterministic operations where repeat requests can create duplicate writes.

Phase 2: Maintainability

1. Split `StudentCourse.jsx` into:
 - class data hook;
 - content/overview views;
 - announcement module;
 - assessment runner;

- submission dialog;
 - attempt results/history.
2. Split `ClassDetail.jsx` into:
 - people manager;
 - content manager;
 - assessment builder;
 - response/grading view;
 - gradebook;
 - activity/settings.
 3. Break `firestoreService.js` into domain modules:
 - users;
 - sectors/templates/classes;
 - enrollments;
 - content;
 - assessments;
 - submissions/grades;
 - notifications/logging.
 4. Introduce TypeScript or runtime schema validation.
 5. Define shared constants/types for roles, statuses, question types, and paths.
 6. Remove legacy/dead fields after a documented migration.
 7. Replace mojibake/encoding-damaged symbols with UTF-8 text or Lucide icons.

Phase 3: Data architecture

1. Publish a versioned Firestore schema.
2. Add schema version fields and idempotent migration scripts.
3. Define authoritative ownership for duplicated class/enrollment/member data.
4. Calculate grades/progress on the backend.
5. Add export/backup and restore drills.
6. Introduce retention policies for logs, submissions, and deleted users.
7. Add audit fields consistently: `createdAt`, `createdBy`, `updatedAt`, `updatedBy`.

Phase 4: UX, accessibility, and performance

1. Test all authenticated screens at 320px and with 200% zoom.
2. Ensure all icon-only buttons have accessible labels and consistent hit targets.
3. Replace horizontal tables on phones with cards or responsive detail rows.
4. Add keyboard and screen-reader tests for modals, builders, and quizzes.
5. Virtualize or paginate large people, notification, and activity lists.
6. Reduce broad real-time listeners and unsubscribe reliably.
7. Profile Core Web Vitals and large Firebase bundles.

8. Add loading skeletons and recoverable error states.

Phase 5: Product and operations

1. Define grading policy, attempt policy, late-work policy, and completion criteria with stakeholders.
 2. Add operational dashboards for Function failures and permission denials.
 3. Configure budgets and billing alerts.
 4. Document support escalation and incident-response ownership.
 5. Add release notes, version tags, and a change log.
 6. Establish data privacy, retention, and account-deletion procedures.
-

13. Troubleshooting

13.1 App says Firebase configuration is missing

Check:

- The correct `.env.*.local` file exists in `HYTech/`.
- Every required `VITE_FIREBASE_*` value is populated.
- The dev server was restarted after environment changes.
- The command uses the intended Vite mode.

13.2 “Missing or insufficient permissions”

Check:

1. Firebase Auth user is signed in.
2. `users/{uid}` exists.
3. Role and status values match rules.
4. Student email verification/admin-created exception is correct.
5. The class ID and trainer/co-trainer ownership are correct.
6. Latest rules were deployed to the same project used by the frontend.
7. Browser requests are not pointing at production while rules were deployed to staging.

Do not weaken rules globally to make an error disappear.

13.3 Callable Function is not found

Check:

- The frontend uses Functions region `asia-southeast1`.
- The Function is exported from `functions/src/index.js`.
- Functions were deployed after the code change.
- The client and deployed project IDs match.
- Billing/runtime deployment completed successfully.

13.4 Quiz submission fails

Check:

- The assessment is published and accepting responses.
- Availability/due date and time-limit fields are valid.
- The student has an active enrollment.
- Required question types produce the expected answer shape.
- The item may live under either `assessments` or `assignments`.
- `submitAssessmentAttempt` logs show the exact `HttpError` code.
- The latest submission Function and rules were deployed together.

13.5 Trainer cannot see or grade a manual response

Check:

- Attempt status is `pending_review`.
- `requiresManualGrading` is true.
- The caller is an active class trainer, co-trainer, or admin.
- `gradeAssessmentAttempt` is deployed.
- The response is read from the correct parent collection.

13.6 File upload fails

Check:

- Auth, role, and class membership.
- File size and permitted MIME type.
- Storage path matches `storage.rules`.
- CORS or App Check enforcement.
- Storage bucket environment value.
- SVG avatars are intentionally rejected.

13.7 Playwright refuses to run

The test runner intentionally accepts only:

- `hytech-lms-staging.web.app`, OR
- `localhost` with `E2E_ALLOW_LOCAL=true`.

Check `.env.e2e.local` and `playwright.config.js`.

13.8 Production UI did not change

Check:

- `npm run build` completed.
 - Firebase Hosting deployed the `dist` folder.
 - The correct project was targeted.
 - `index.html` is not being served by a stale proxy.
 - The expected commit is in the deployment workflow.
 - Backend-dependent changes may also require Functions/rules deployment.
-

14. Coding and review conventions

14.1 Change discipline

- Keep changes scoped and reversible.
- Never mix an unrelated dependency upgrade into a feature fix.
- Preserve user data and unrelated working-tree changes.
- Validate document IDs and resolved paths before destructive operations.
- Use transactions/batches for multi-document consistency.
- Avoid browser-side writes for authoritative security or grading data.

14.2 Pull-request checklist

- ☐ Requirement and acceptance criteria are stated.
- ☐ Security impact is reviewed.
- ☐ Staging environment was used.
- ☐ Production build passes.
- ☐ Relevant Playwright tests pass.
- ☐ Rules/Functions tests pass for backend changes.
- ☐ Mobile, tablet, and desktop were checked.
- ☐ Keyboard and labels were checked for UI changes.
- ☐ Migration and rollback are documented.
- ☐ No credentials, personal data, or internal audit files were added.
- ☐ Functions/rules/indexes are included when the frontend depends on them.
- ☐ Post-deployment verification steps are included.

14.3 Definition of done

A change is not complete when it only works in the browser locally. It is done when:

- Security Rules and Cloud Functions support the behavior.
 - Failure states are understandable.
 - Tests cover the risky path.
 - Staging verification passes.
 - Deployment order is documented.
 - Monitoring and rollback are possible.
-

15. Operational ownership checklist

The receiving team should identify named owners for:

Area	Required owner
Product requirements and grading policy	Product/institution representative
GitHub repository and branch protection	Engineering lead
Firebase production project	Platform owner
Firebase staging project	QA/platform owner
Billing and budgets	Organization owner
Authentication and user lifecycle	Application owner
Security Rules and App Check	Security/backend owner
Backups and data recovery	Platform/data owner
CI/CD credentials	DevOps owner
Incident response and privacy requests	Security/privacy owner

Record contacts in a restricted operational system, not in this repository.

16. First-week plan for a receiving developer

Day 1: Access and read-only orientation

- Clone the repository.
- Read this guide and active Firebase/QA documents.

- Inspect routes, rules, and Functions.
- Confirm staging access.
- Do not deploy.

Day 2: Reproduce the system

- Install dependencies.
- Configure staging environment files.
- Build and run the app.
- Sign in using dedicated QA roles.
- Complete the basic class and assessment flow.

Day 3: Verify security boundaries

- Run Rules tests and negative role tests.
- Confirm answer keys are private.
- Confirm cross-user and cross-class writes fail.
- Review App Check metrics and CI authentication.

Day 4: Verify releases

- Run the full QA suite.
- Review the deploy dry-run.
- Document the exact current deployment and rollback process.
- Do not test deployment against production without approval.

Day 5: Start remediation

- Tackle P0/P1 items from the roadmap.
- Add regression tests before refactoring large components.
- Establish branch protection and a QA-gated release process.

17. Final handover notes

The application has substantial working functionality and a clear Firebase foundation. Its largest risks are not the visual interface; they are trust boundaries, deployment governance, confidential material exposure, and maintainability concentrated in very large files.

Future developers should preserve these principles:

1. **Staging first.**
 2. **Rules and backend validation are the security boundary.**
 3. **Grades, attempts, and progress must be authoritative server data.**
 4. **No production deployment without passing QA and a rollback plan.**
 5. **No credentials or sensitive reports in Git.**
 6. **Refactor with regression tests, not by rewriting the system blindly.**
-

Appendix A: Command reference

Run from **HYTech/**.


```
# Install
npm ci
npm ci --prefix functions
npx playwright install chromium

# Local development
npm run dev
npm run dev:staging

# Builds
npm run build
npm run build:staging
npm run preview

# Tests
npm run test:e2e
npm run test:e2e:public
npm run test:e2e:responsive
npm run test:e2e:roles
npm run test:qa:full

# QA data
npm run qa:seed:identities
npm run qa:credentials
npm run qa:seed:data
npm run qa:verify-seed
npm run qa:cleanup

# Firebase inspection
firebase projects:list
firebase use
firebase functions:log --project staging

# Validation
node --check functions/src/index.js
firebase deploy --project staging --dry-run

# Staging resources
firebase deploy --only firestore:rules,firestore:indexes --project staging
firebase deploy --only storage --project staging
firebase deploy --only functions --project staging
firebase deploy --only hosting --project staging
```

Appendix B: Environment-variable reference

Variable	Used by	Sensitivity/notes
VITE_FIREBASE_API_KEY	Browser	Public identifier; restrict in Google Cloud
VITE_FIREBASE_AUTH_DOMAIN	Browser	Firebase Auth domain

Variable	Used by	Sensitivity/notes
VITE_FIREBASE_PROJECT_ID	Browser	Must match intended environment
VITE_FIREBASE_STORAGE_BUCKET	Browser	Environment-specific bucket
VITE_FIREBASE_MESSAGING_SENDER_ID	Browser	Firebase web configuration
VITE_FIREBASE_APP_ID	Browser	Firebase web-app identifier
VITE_RECAPTCHA_SITE_KEY	Browser	Public App Check site key
VITE_APPCHECK_DEBUG_TOKEN	Local only	Sensitive; never commit
E2E_BASE_URL	Playwright	Staging URL or guarded localhost
E2E_*_EMAIL	Playwright	Staging-only account
E2E_*_PASSWORD	Playwright	Secret; never commit
FIREBASE_SERVICE_ACCOUNT_JSON	QA automation	Highly sensitive
FIREBASE_TOKEN	Legacy deploy workflow	Deprecated long-lived secret; replace

Appendix C: Useful source locations

Concern	Source
Route definitions	HYTech/src/App.jsx
Firebase client	HYTech/src/firebase.js
Auth state	HYTech/src/context/AuthContext.jsx
Browser data layer	HYTech/src/utils/firestoreService.js
Student class experience	HYTech/src/components/student/StudentCourse.jsx
Trainer class management	HYTech/src/components/trainer/ClassDetail.jsx
Cloud Functions	HYTech/functions/src/index.js
Firestore authorization	HYTech/firestore.rules
Storage authorization	HYTech/storage.rules
Hosting and headers	HYTech/firebase.json
Firestore indexes	HYTech/firestore.indexes.json
Test configuration	HYTech/playwright.config.js
CI QA	.github/workflows/qa.yml

Concern	Source
CI deployment	<code>.github/workflows/deploy.yml</code>